

Homework #2

November 11, 2020

Chun-Yuan (Scott) Chiu
chunyuac@andrew.cmu.edu

1.

(a)

Below is the definition of the function `data` that, given the value of ρ , will draw a sample of size 100 from the bivariate normal distribution with desired mean and covariance matrix.

```
[1]: from scipy.stats import multivariate_normal
from pandas import DataFrame, Series
import matplotlib.pyplot as plt

sample_size = 100
mu = [1, 2]
sigma = lambda rho: [[1, rho], [rho, 1]]
dist = lambda rho: multivariate_normal(mean=mu, cov=sigma(rho))
data = lambda rho: DataFrame(dist(rho).rvs(sample_size), columns=['x', 'y'])
mean = DataFrame([mu], columns=['x', 'y'])
```

Use the `data` function to draw a sample with $\rho = 0.6$. The size of the resulting sample is confirmed to be 100×2 .

```
[2]: X = data(rho=0.6)
X.shape
```

```
[2]: (100, 2)
```

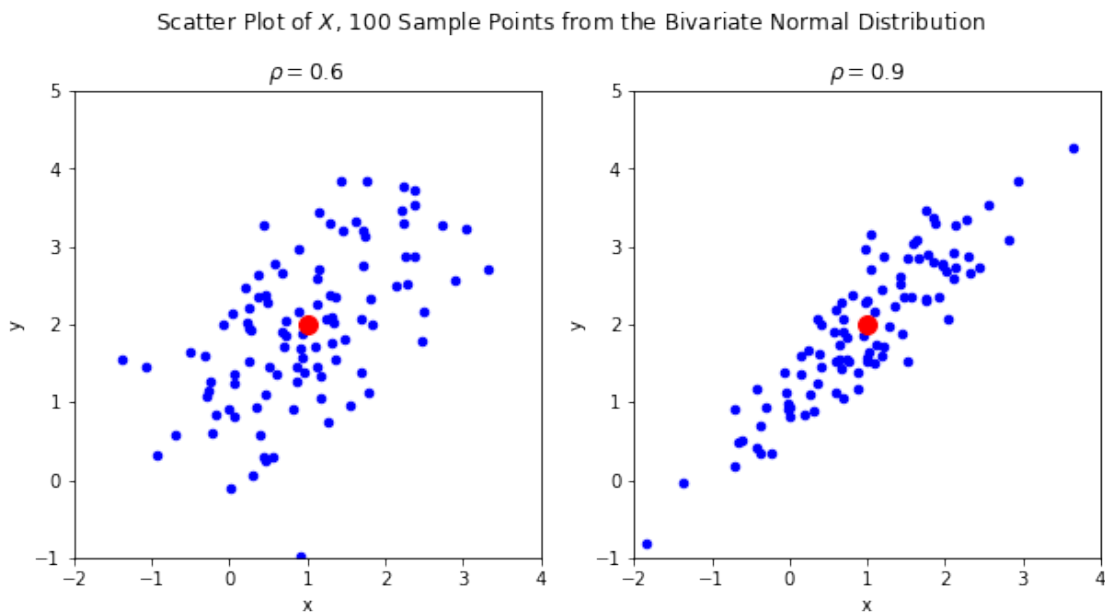
Here is a comparison of the scatter plots with $\rho = 0.6$ and $\rho = 0.9$. Both look like an ellipse-shaped cloud of data points, but the ellipse is more flattened when ρ is large.

```
[3]: fig, (ax0, ax1) = plt.subplots(1, 2)

X = data(rho=0.6)
X.plot(kind='scatter', x='x', y='y', color='blue', ax=ax0)
data(rho=0.9).plot(kind='scatter', x='x', y='y', color='blue', ax=ax1)
```

```
mean.plot(kind='scatter', x='x', y='y', color='red', s=100, xlim=(-2, 4),
    →ylim=(-1, 5), ax=ax0)
mean.plot(kind='scatter', x='x', y='y', color='red', s=100, xlim=(-2, 4),
    →ylim=(-1, 5), ax=ax1)

ax0.set(aspect=1, title='$\\rho = 0.6$')
ax1.set(aspect=1, title='$\\rho = 0.9$');
fig.set_size_inches(10, 5)
fig.suptitle('Scatter Plot of $X$, 100 Sample Points from the Bivariate Normal
    →Distribution')
plt.show()
```



(b)

Below is the histogram of the marginal distribution. The blue line is the KDE plot and the red line is the normal pdf, the height scaled to match the sample size.

```
[4]: from scipy import stats
import numpy as np
import seaborn as sns

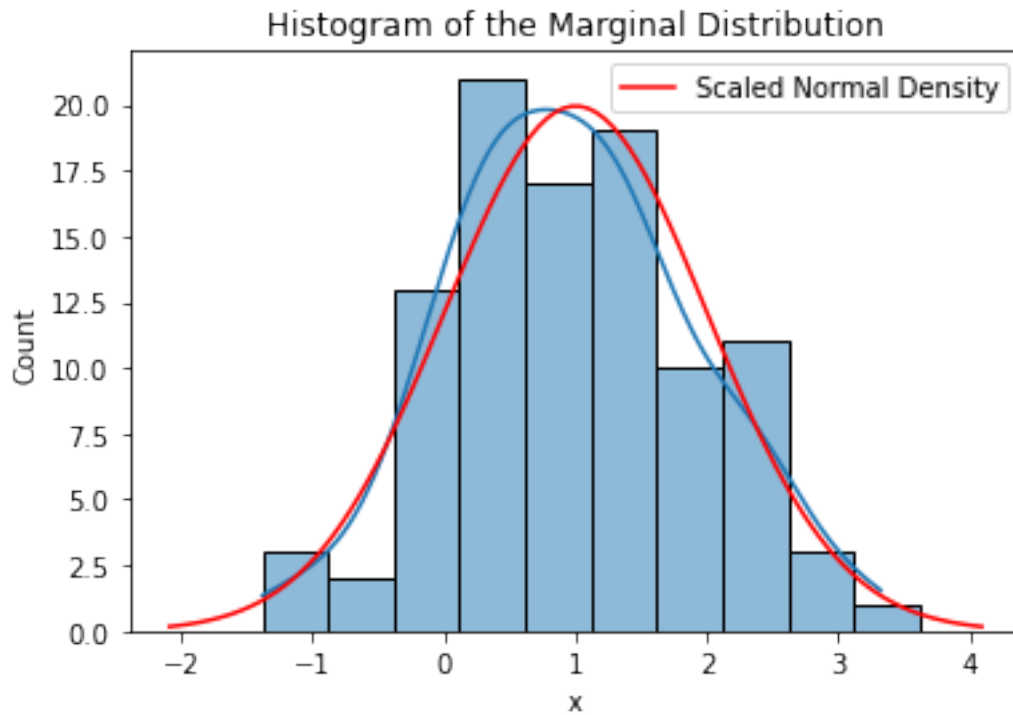
rho = 0.6
norm = stats.norm(loc=mu[0], scale=sigma(rho)[0][0])

binwidth = 0.5
```

```

xs = np.linspace(norm.ppf(0.001), norm.ppf(0.999), 100)
ax = sns.histplot(data=X, x='x', kde=True, binwidth=binwidth)
ax.set(title='Histogram of the Marginal Distribution')
Series({x: binwidth*sample_size*norm.pdf(x) for x in xs}, name='Scaled Normal_
→Density').plot(color='red', ax=ax, legend=True)
plt.show()

```



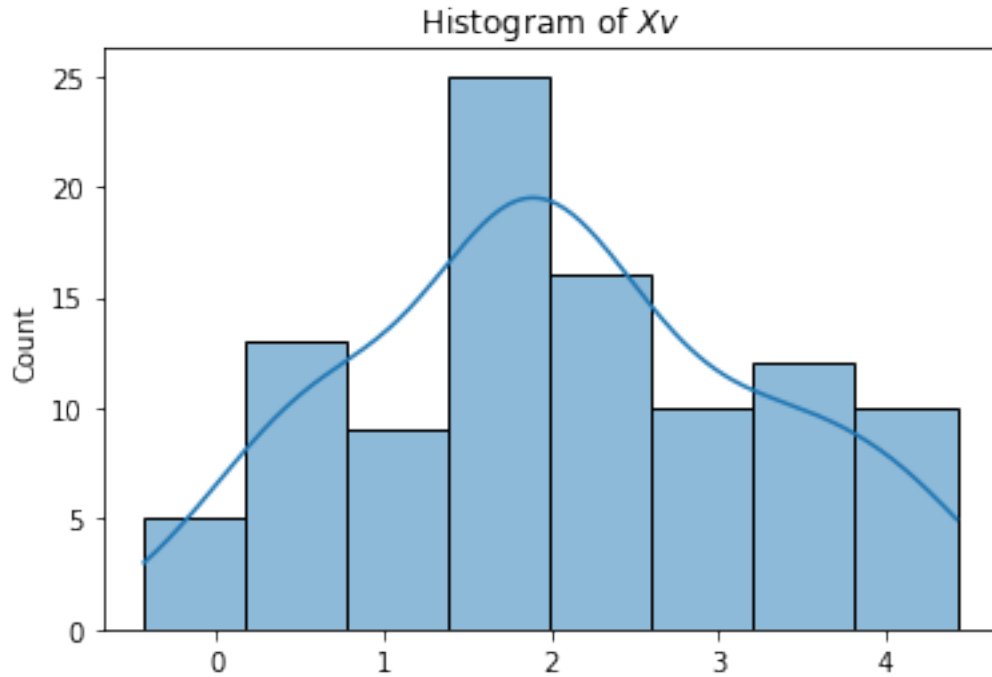
(c)

Below is a histogram of Xv . The variance looks larger than the first column of the data. In fact analytically it can be shown that $V(Xv) = 1 + \rho$.

```

[5]: ax = sns.histplot(X @ [1/np.sqrt(2), 1/np.sqrt(2)], kde=True)
ax.set(title='Histogram of $Xv$')
plt.show()

```



(d)

The eigenvalues and eigenvectors are computed by the `numpy.linalg.eig` function. The eigenvectors are in the major and minor axis directions of the ellipse. They also are the directions in which the data movements have the largest and smallest variances. The variances are proportional to $2\sqrt{\lambda_i}$.

```
[6]: fig, (ax0, ax1) = plt.subplots(1, 2)

data(rho=0.6).plot(kind='scatter', x='x', y='y', color='blue', ax=ax0)
data(rho=0.9).plot(kind='scatter', x='x', y='y', color='blue', ax=ax1)
mean.plot(kind='scatter', x='x', y='y', color='red', s=100, xlim=(-2, 4),
→ylim=(-1, 5), ax=ax0)
mean.plot(kind='scatter', x='x', y='y', color='red', s=100, xlim=(-2, 4),
→ylim=(-1, 5), ax=ax1)

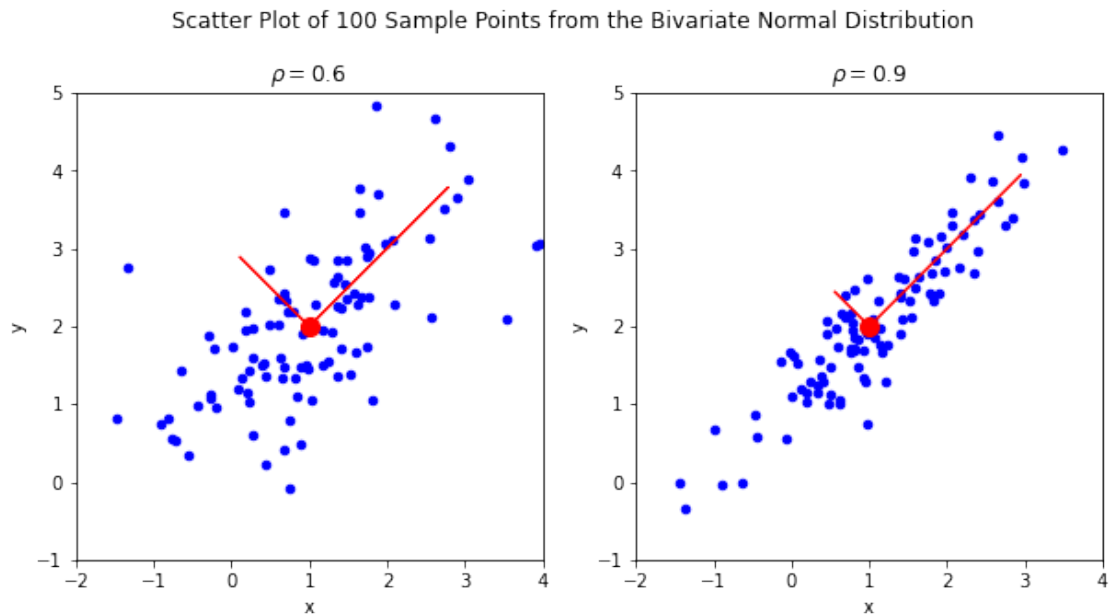
w, v = np.linalg.eig(sigma(0.6))
ax0.arrow(*mu, *(2*np.sqrt(w[0])*v[:, 0]), color='r')
ax0.arrow(*mu, *(2*np.sqrt(w[1])*v[:, 1]), color='r')

w, v = np.linalg.eig(sigma(0.9))
ax1.arrow(*mu, *(2*np.sqrt(w[0])*v[:, 0]), color='r')
ax1.arrow(*mu, *(2*np.sqrt(w[1])*v[:, 1]), color='r')
```

```

ax0.set(aspect=1, title='$\\rho = 0.6$')
ax1.set(aspect=1, title='$\\rho = 0.9$');
fig.set_size_inches(10, 5)
fig.suptitle('Scatter Plot of 100 Sample Points from the Bivariate Normal_
↪Distribution')
plt.show()

```



(g)

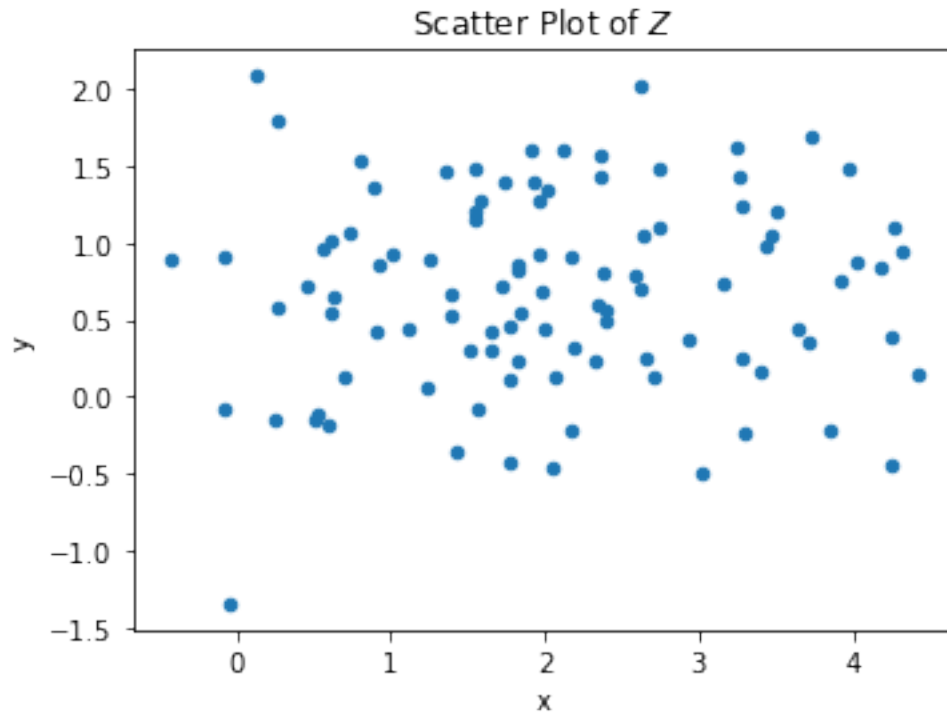
Again the below scatter plot of Z looks like an ellipse-shaped cloud of data points, only this ellipse is not rotated. Its major and minor axes are horizontal and vertical.

```

[7]: w, v = np.linalg.eig(sigma(0.6))
v1 = v[:, 0]
v2 = v[:, 1]

Z = DataFrame({'x': X@v1, 'y': X@v2})
ax = Z.plot(kind='scatter', x='x', y='y')
ax.set(aspect=1, title='Scatter Plot of $Z$')
plt.show()

```



The correlations are computed below. For X it is close to 0.6 which is expected. For Z it is close to zero.

```
[8]: print(X['x'].corr(X['y'])) # X
      print(Z['x'].corr(Z['y'])) # Z
```

```
0.5619014462466392
0.05885194747015292
```

2.

(a)

```
[9]: from pandas import Series
      import seaborn as sns

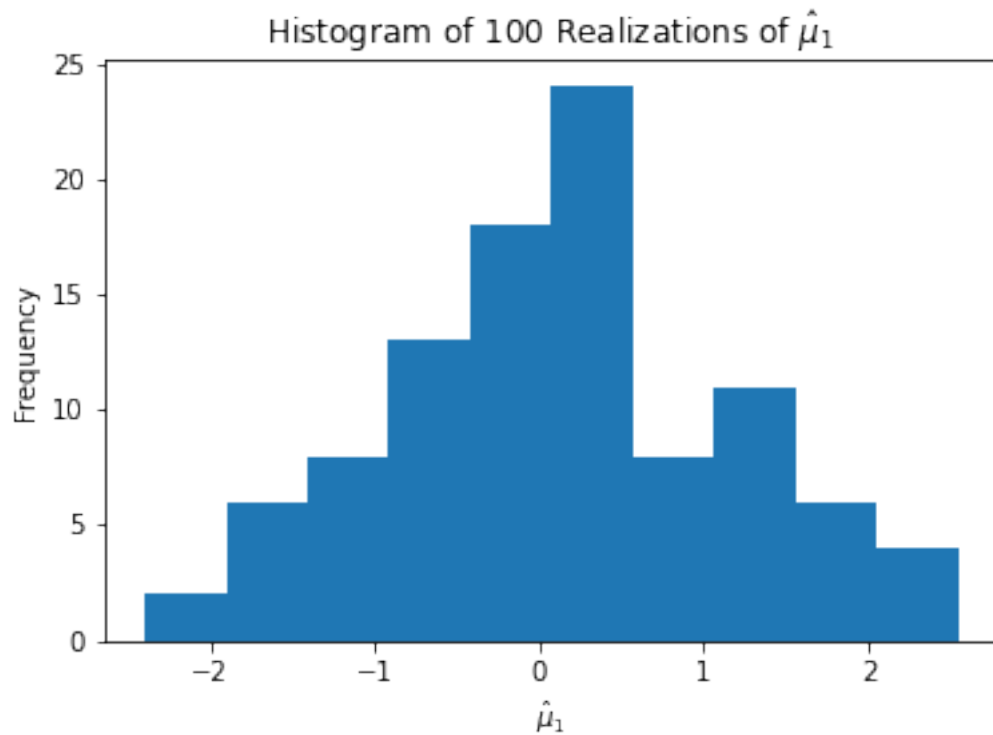
      sample = [stats.norm.rvs(size=100)[0] for i in range(100)]
      ss = Series(sample)
      ax = ss.plot(kind='hist')
      ax.set(xlabel='$\hat{\mu}_1$', title='Histogram of 100 Realizations of $\hat{\mu}_1$')
      plt.show()
```

```

z = stats.norm.ppf(1 - 0.025)    # 1.96

print('Fraction of confidence intervals that cover 0:', ss[(ss-z < 0) & (ss+z >=
↪0)].size / ss.size)

```



Fraction of confidence intervals that cover 0: 0.94

(b)

```

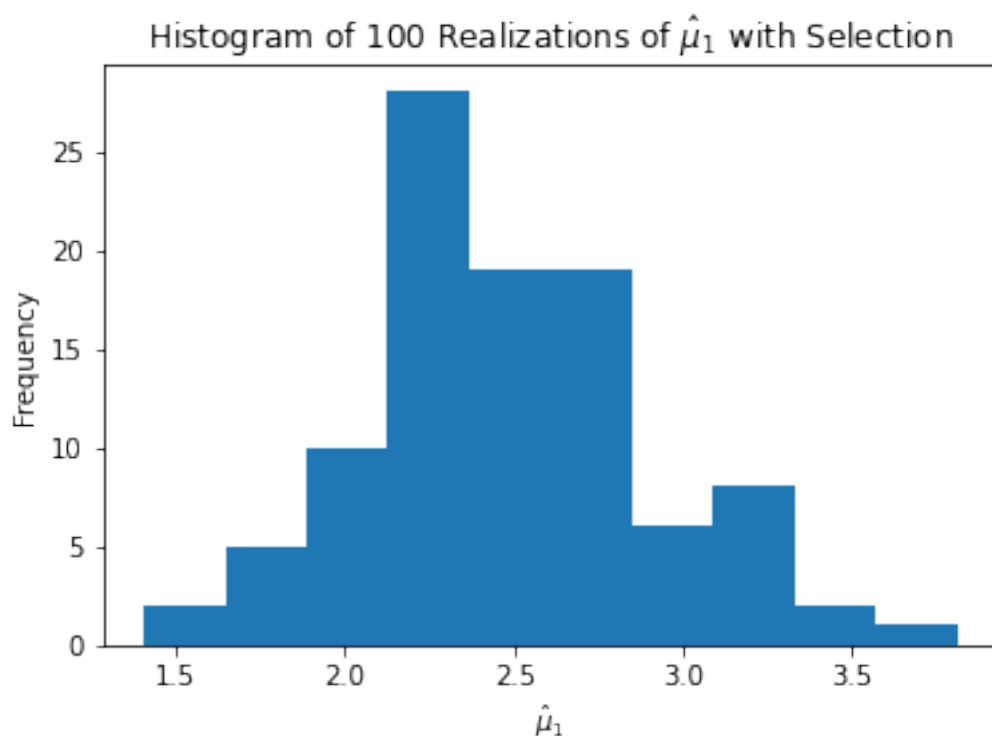
[10]: from pandas import Series

sample = [max(stats.norm.rvs(size=100)) for i in range(100)]
ss = Series(sample)
ax = ss.plot(kind='hist')
ax.set(xlabel='$\hat{\mu}_1$', title='Histogram of 100 Realizations of ↪
↪ $\hat{\mu}_1$ with Selection')
plt.show()

z = stats.norm.ppf(1 - 0.025)    # 1.96

```

```
print('Fraction of confidence intervals that cover 0:', ss[(ss-z < 0) & (ss+z > 0)].size / ss.size)
```



Fraction of confidence intervals that cover 0: 0.1

3.

(a)

The proof will consist of two parts. First we show that $\hat{\beta}^{\text{ridge}}$ is the solution to $(X^T X + \lambda I)\beta = X^T y$. Second we show that the OLS estimator with predictor matrix $\tilde{X}$ and $\tilde{y}$ is also the solution to the same linear system. Thus they are identical.

To verify the first part, we expand the objective function:

$$\begin{aligned}
 \|y - X\beta\|_2^2 + \lambda\|\beta\|_2^2 &= (y - X\beta)^T(y - X\beta) + \lambda\beta^T\beta \\
 &= (y^T y - 2\beta^T X^T y + \beta^T X^T X \beta) + \lambda\beta^T\beta \\
 &= y^T y - 2\beta^T X^T y + \beta^T (X^T X + \lambda I)\beta.
 \end{aligned}$$

This is a quadratic form in β . At its minimum the gradient must be zero, so

$$-2X^T y + ((X^T X + \lambda I) + (X^T X + \lambda I)^T)\beta = 0.$$

Simplify to conclude that $\hat{\beta}^{\text{ridge}}$, where the objective function reaches its minimum, is the solution to $(X^T X + \lambda I)\beta = X^T y$.

For the second part, we know that the OLS estimator is the solution to $\tilde{X}^T \tilde{X} \beta = \tilde{X}^T \tilde{y}$. Using block matrix multiplication, it is easy to see that $\tilde{X}^T \tilde{X} = X^T X + \lambda I$ and $\tilde{X}^T \tilde{y} = X^T y$. So again the OLS estimator is the solution to $(X^T X + \lambda I)\beta = X^T y$ and hence the proof is complete.

(b)

If the columns of $\tilde{X}$ are linearly dependent, by definition there exists a non-zero vector $v \in \mathbb{R}^p$ such that $\tilde{X}v$ is a zero vector in $\mathbb{R}^{n+p}$. But using block matrix multiplication, we see that this also means $(\sqrt{\lambda}I)v = \sqrt{\lambda}v$ is a zero vector in $\mathbb{R}^p$, a contradiction. So the columns of $\tilde{X}$ are linearly independent.

(c)

In part (a), we have established that $\hat{\beta}^{\text{ridge}}$ is the solution to $(X^T X + \lambda I)\beta = X^T y$. Assuming $\lambda > 0$, or $\lambda = 0$ but $n > p$ and X is full-rank, $(X^T X + \lambda I)$ is clearly positive definite and hence invertible, in which case $\hat{\beta}^{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y$, and $a^T \hat{\beta}^{\text{ridge}} = a^T (X^T X + \lambda I)^{-1} X^T y$ is a linear function of y . Without the assumptions, $\hat{\beta}^{\text{ridge}}$ is still the solution to $(X^T X + \lambda I)\beta = X^T y$ but there is no guarantee that $X^T X + \lambda I$ is invertible.

4.

(a)

Note that $U^T U = I$ and, because D is diagonal, $D(D^2)^{-1}D = I$. Thus in OLS we have

$$\begin{aligned} \hat{Y} &= X(X^T X)^{-1} X^T Y \\ &= U D V^T (V D U^T U D V^T)^{-1} V D U^T Y \\ &= U D V^T (V D^2 V^T)^{-1} V D U^T Y \\ &= U D V^T (V^T)^{-1} (D^2)^{-1} V^{-1} V D U^T Y \\ &= U D (D^2)^{-1} D U^T Y \\ &= U U^T Y. \end{aligned}$$

To interpret this result, note that $U^T z$ is the length of the projection of z onto the linear space spanned by columns of U , so $U U^T z$ is the actual projection vector. Since $X = U D V^T$ and both D and V are invertible, the linear space spanned by columns of U and the one spanned by columns of X are the same. Thus we obtain the same geometric interpretation of linear regression we saw in class.

(b)

In the ridge regression we have

$$\begin{aligned}
\hat{Y}_{ridge} &= X(X^T X + \lambda I)^{-1} X^T Y \\
&= U D V^T (V D U^T U D V^T + \lambda I)^{-1} V D U^T Y \\
&= U D V^T (V D^2 V^T + \lambda I)^{-1} V D U^T Y \\
&= U D V^T (V D^2 V^T + \lambda V V^T)^{-1} V D U^T Y \\
&= U D V^T (V (D^2 + \lambda I) V^T)^{-1} V D U^T Y \\
&= U D V^T (V^T)^{-1} (D^2 + \lambda I)^{-1} V^{-1} V D U^T Y \\
&= U D (D^2 + \lambda I)^{-1} D U^T Y \\
&= U D^2 (D^2 + \lambda I)^{-1} U^T Y.
\end{aligned}$$

In matrix form this is the most simplified expression we can get. Since D is diagonal, we have

$$D^2 (D^2 + \lambda I)^{-1} = \text{diag} \left(\frac{d_1^2}{d_1^2 + \lambda}, \frac{d_2^2}{d_2^2 + \lambda}, \dots, \frac{d_p^2}{d_p^2 + \lambda} \right).$$

(c)

In (b) we have established that $\hat{Y}_{ridge} = U S U^T Y$, where

$$S = D^2 (D^2 + \lambda I)^{-1} = \text{diag} \left(\frac{d_1^2}{d_1^2 + \lambda}, \frac{d_2^2}{d_2^2 + \lambda}, \dots, \frac{d_p^2}{d_p^2 + \lambda} \right).$$

From this $\hat{Y}_{ridge}$ formula we can already see that when $\lambda = 0$, we will have $S = I$ and $\hat{Y}_{ridge}$ reduces to the OLS estimator $\hat{Y}$.

Next we will prove

$$\hat{Y}_{ridge} = \sum_{j=1}^p \frac{d_j^2}{d_j^2 + \lambda} u_j u_j^T Y.$$

First denote $s_j = d_j^2 / (d_j^2 + \lambda)$, and denote by S_j the matrix $\text{diag}(0, 0, \dots, 0, s_j, 0, \dots, 0)$. Then S can be written as a sum $S = \sum_{j=1}^p S_j$ and thus

$$\hat{Y}_{ridge} = \sum_{j=1}^p U S_j U^T Y.$$

But using block matrix multiplication,

$$\begin{aligned}
US_jU^T &= (u_1 \ u_2 \ \dots \ u_p) \begin{pmatrix} 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & \dots & 0 & s_j & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \end{pmatrix} U^T \\
&= (\vec{0} \ \vec{0} \ \dots \ \vec{0} \ s_j u_j \ \vec{0} \ \dots \ \vec{0}) \begin{pmatrix} u_1^T \\ u_1^T \\ \vdots \\ u_p^T \end{pmatrix} \\
&= s_j u_j u_j^T = \frac{d_j^2}{d_j^2 + \lambda} u_j u_j^T.
\end{aligned}$$

The proof is complete.

If $\lambda > 0$, it has a larger impact to the scalar $d_j^2/(d_j^2 + \lambda)$ when d_j is small. When d_j^2 is very large compared to λ , the shrinkage almost does nothing. As we have seen in question 1(d), the variance of data movements in the v_j direction is proportional to d_j (in 1(d) we were looking at the eigen decomposition of the covariance matrix of data, which is proportional to $X^T X$, while here we are looking at the SVD of X , that is $X = UDV^T$, but then $X^T X = VD^2V^T$ is the eigen decomposition of $X^T X$, so d_j is proportional to the square root of eigen values computed in 1(d)). So the ridge regression is equivalent to constructing new variables by PCA (the PCA directions), and then shrinking data along PCA directions, and it prefers to shrink the data more along the directions that have smaller variances, and to shrink the data less along the directions that have larger variances. That is, it tends ignore the data movements in the directions that have smaller variances, making those movements less impactful in prediction, which is a sensible thing to do. When we use PCA for dimensionality reduction, we only take the first few PCA directions that have larger variances. The ridge regression takes all but shrinks their impact in prediction differently according to the variances.